

Einsendeaufgabe Softwaretechnik

Refactoring E1

4. Semester SoSe26

Eingereicht von: Marvin Bock, 106334

Kurs: Softwaretechnik (BHT MIB 20 S26)

Kursleitung: Prof. Dr.-Ing. Stefan Edlich

Refactoring-Katalog

Ich finde die „einfachen“ Refactorings beeindruckend, welche den Code klarer und ordentlicher gestalten.

„Splitt Loop“ ist eine dieser Methoden. Oftmals sind Aufgabenstellungen mit mehreren Schleifen in einander verbaut, sodass diese sehr schnell unübersichtlich werden und der Code kaum nachvollziehbar wirkt.

Auch „Extract Superclass“ fand ich interessant. Da dabei gemeinsame Eigenschaften und Methoden mehrerer Klassen in eine Oberklasse ausgelagert werden. Dadurch wird doppelter Code vermieden und die Struktur des Programmes übersichtlicher. Es erleichtert die spätere Erweiterung. Dies konnte ich bereits in Grundlagen der Programmierung 2 lernen.

Ein Refactoring, welches ich schwieriger nachvollziehen konnte, war „Substitute Algorithm“. Die Grundidee, einen bestehenden Algorithmus vollständig durch einen anderen zu ersetzen, ist zwar verständlich, allerdings war für mich nicht immer klar, ab wann dies noch als Refactoring zählt und nicht bereits eine größere Änderung am Programm darstellt. Im Vergleich zu den anderen Refactorings wirkte diese Methode deutlich abstrakter. Zusätzlich fallen mir solche abstrakteren Refactorings und moderne Kurzschreibweisen wie Lambda- bzw. Arrow-Funktionen teilweise noch schwer, da der Ablauf des Codes dadurch für mich nicht immer direkt nachvollziehbar ist.

Ein Refactoring, welches ich persönlich eher für überflüssig halte, ist „Rename Variable“. Meiner Meinung nach sollten Variablen bereits während der Entwicklung eindeutig und verständlich benannt werden. Dadurch würde ein späteres Umbenennen größtenteils entfallen. Trotzdem kann dieses Refactoring hilfreich sein, wenn sich Anforderungen oder Bedeutungen im Verlauf eines Projektes ändern.

Welche IDE wird genutzt: Eclipse

Zur Verbesserung habe ich ein alten Code aus dem Modul Grundlagen der Programmierung genommen. Ziel war es „Geschenkpapier“ zu drucken. Alle Java-Dateien werden mit hochgeladen. Das Refactoring folgt in der Geschenkpapier.java.

Code Vorher:

```
package Geschenkpapier;
/**
 * Dateiname: Geschenkpapier.java
 * Beschreibung: Eine Klasse zur Erstellung eines Musters in der Ausgabe mit Vorgaben vom Benutzer
 *
 * @author Marvin B. (mabo3343@bht-berlin.de)
 * @version 1.0, 12/2024
 * @param Zeilen = Anzahl der gewünschten Zeilen, spalten = Anzahl der gewünschten Spalten, art = Art des Musters Case M1
oder M2
 */

public class Geschenkpapier {

    public static void muster (String art, int zeilen, int spalten) {

        switch (art) {
            //Muster 1 ***
            case "M1" -> {
                //Zeilen schleife
                for (int z=1; z <= zeilen; z++) {
                    // Spalten schleife
                    for (int i=1; i <= spalten; i++) {
                        System.out.print("*** ");
                    }
                    System.out.println();
                }
            }
            //Muster 2
            case "M2" -> {
                int z = 1;
                int s = 1;
                int s2 = 1;
                //Zeilen schleife
                while (zeilen >= z) {
                    //If Unterscheidung gerade Zeile oder ungerade Zeile
                    if (z % 2 == 0) {
                        //Spalten schleife
                        while (spalten >= s) {
                            System.out.print(":-");
                            s++;
                        }
                    } else {
                        while (spalten >= s2){
                            System.out.print(":- ");
                            s2++;
                        }
                    }
                    System.out.println();
                    z++;
                    s = 1;
                    s2 = 1;
                }
            }
        }
    }
}
```

Die Test.Java:

```
package Geschenkpapier;

import java.util.Scanner;
/**
 * Dateiname: GeschenkpapierTest.java
 * Beschreibung: Ein Test Programm zum testen der Klasse Geschenkpapier.
 *
 * @author Marvin B. (mabo3343@bht-berlin.de)
 * @version 1.0, 12/2024
 */
public class GeschenkpapierTest {

    public static void main(String[] args) {

        //Variablen
        int spalten;
        int zeilen;
        String muster;
        String yn = "y";
        Scanner tastatur;
        tastatur = new Scanner(System.in);

        //Muster Auswahl
        System.out.println("Sie können zwischen Muster 1: '*' mit 'M1' \noder \nMuster 2: ':-)' / '-:' mit 'M2' wählen");
        //Start Programmschleife
        do {
            System.out.print("Bitte wählen Sie jetzt zwischen 'M1' und 'M2': ");
            muster = tastatur.next();
            // If Prüfung der Eingabe
            if (!muster.equals("M1") && !muster.equals("M2")) {
                System.out.println("fehlerhafte Eingabe");
                //Schleifenabbruch bei fehlerhafter Eingabe
                continue;
            }
        }
        //Eingabe der Spalten und Zeilen
        System.out.print("Geben Sie nun die Anzahl der Spalten und der Zeilen an\nSpaltenanzahl: ");
        spalten = tastatur.nextInt();

        System.out.print("Zeilenanzahl: ");
        zeilen = tastatur.nextInt();

        //Aufruf Klasse Geschenkpapier und übergabe der Parameter, Musterart, Zeilenanzahl, Spaltenanzahl
        Geschenkpapier.muster(muster, zeilen, spalten);

        //Wiederholung des Programmes gewünscht?
        System.out.println("Möchten sie das Programm wiederholen? y/n");
        yn = tastatur.next();
        } while (yn.equals("y"));

        //Scanner schließen
        tastatur.close();
    }
}
```

Code Nachher:

```
package Geschenkpapier;

/**
 * Dateiname: Geschenkpapier.java
 * Beschreibung: Eine Klasse zur Erstellung eines Musters in der Ausgabe mit Vorgaben vom Benutzer
 *
 * @author Marvin B.
 * @version 1.1, 06/2026
 */

public class Geschenkpapier {

    public static void muster(String art, int zeilen, int spalten) {

        switch (art) {
            case "M1" -> druckeMuster1(zeilen, spalten);
            case "M2" -> druckeMuster2(zeilen, spalten);
        }
    }

    public static void druckeMuster1(int zeilen, int spalten) {
        // Zeilen schleife
        for (int z = 1; z <= zeilen; z++) {
            // Spalten schleife
            for (int i = 1; i <= spalten; i++) {
                System.out.print("*** ");
            }
            System.out.println();
        }
    }

    public static void druckeMuster2(int zeilen, int spalten) {
        int z = 1;
        int s = 1;
        int s2 = 1;
        // Zeilen schleife
        while (zeilen >= z) {
            // If Unterscheidung gerade Zeile oder ungerade Zeile
            if (z % 2 == 0) {
                // Spalten schleife
                while (spalten >= s) {
                    System.out.print(")-.");
                    s++;
                }
            } else {
                while (spalten >= s2) {
                    System.out.print(":- ");
                    s2++;
                }
            }
            System.out.println();
            z++;
            s = 1;
            s2 = 1;
        }
    }
}
```

Durch das Refactoring habe ich die beiden Muster in die Methoden `druckeMuster1()` und `druckeMuster2()` ausgelagert. Dadurch wurde die ursprüngliche Methode deutlich kürzer und übersichtlicher. Außerdem ist nun klarer erkennbar, welche Methode für welches Muster zuständig ist. Ich finde, dass sich der Code dadurch einfacher lesen und später erweitern lässt.

Refactoring mit AI:

Für das Refactoring mit Hilfe einer Generativen AI habe ich das Fowler-Refactoring „Extract Method“ verwendet. Als Prompt habe ich *„Ich gebe dir ein Code Beispiel vor. Nutzte die Extract Method zum Refactoring des Codes...“* eingegeben. Die AI hat dabei den Bereich zur Ausgabe der Rechnungsdetails in die eigene Methode `printDetails()` ausgelagert. Dadurch wurde die ursprüngliche Funktion kürzer und übersichtlicher gestaltet.

Interessant war, dass die AI das Refactoring minimal anders umgesetzt hat als das ursprüngliche Beispiel von Fowler. Die Funktion wurde außerhalb der Hauptfunktion angelegt und benötigte dadurch zusätzliche Parameter. Das Refactoring hat insgesamt direkt funktioniert.

Code Vorher:

```
function printOwing(invoice) {  
  printBanner();  
  let outstanding = calculateOutstanding();  
  
  //print details  
  console.log(`name: ${invoice.customer}`);  
  console.log(`amount: ${outstanding}`);  
}
```

Code Nachher aus dem Katalog	Code Nachher aus der AI
<pre>function printOwing(invoice) { printBanner(); let outstanding = calculateOutstanding(); printDetails(outstanding); function printDetails(outstanding) { console.log(`name: \${invoice.customer}`); console.log(`amount: \${outstanding}`); } }</pre>	<pre>function printOwing(invoice) { printBanner(); let outstanding = calculateOutstanding(); printDetails(invoice, outstanding); } function printDetails(invoice, outstanding) { console.log(`name: \${invoice.customer}`); console.log(`amount: \${outstanding}`); }</pre>